Levamo (WordPress/PAB) to MySQL Data Pipeline

End-to-End Technical Specification: Data Extraction, Storage Schema, API Protocols, Synchronization Lifecycles & Security Controls

1. System Context & Identification of 'Levamo'

In the Doctor Sarah Hensley AI RAG architecture, **'Levamo'** corresponds to the upstream **WordPress + WooCommerce + PAB (Power Action Breakthrough)** platform hosted at WORDPRESS_URL (e.g., dshstaging.doctorsarahhensley.com). It serves as the primary system of record for customer user profiles, e-commerce purchases, and clinical attachment quiz assessments. To empower the RAG AI agent with instant contextual awareness of a client's relationship history, course enrollments, and attachment style without incurring high-latency external HTTP overhead during real-time chat turns, data is mirrored and synchronized into a local, high-speed MySQL database named **rag_users**.

Key Architectural Objective: Provide sub-millisecond local user context retrieval for the RAG LLM prompt generation engine while maintaining absolute data consistency with WooCommerce and PAB webhooks and scheduled ETL cron syncs.

2. Data Extraction: Entities & Fields Taken from Levamo

Data ingested from Levamo is partitioned across three distinct logical entities: Customers, Orders/Products, and Clinical Assessment Profiles. The table below details every field extracted, its transformation, and its downstream target.

Entity	Source Field (Levamo)	Transformation & Parsing Logic	Target in MySQL
Customer	`id (Integer)`	Cast to string; primary unique user key.	`users.id`
Customer	`first_name, last_name, username`	Concatenated as f'{first} {last}'.strip(). Falls back to username if empty.	`users.name`
Customer	`email (String)`	Lowercased and trimmed. Used for unique constraints and secondary alias identity row.	`users.email`
Order	`id (Integer)`	Captured for narrative logging and audit tracking.	`past_records.summary_notes`
Order	`customer_id or billing.email`	Resolves order ownership. Used to link foreign key to users.id.	`purchases.user_id`
Order Item	`line_items[].name`	Cleaned product title (e.g. *'The Science of Connection'*).	`purchases.item_title`
Order Item	`Title Classifier`	Categorization heuristic: • If 'book' or 'science of' in title → **book** • If 'session', 'coaching', 'minute' → **session** • Otherwise → **course**	`purchases.item_type (ENUM)`
Order	`date_created`	Parsed to YYYY-MM-DD timestamp.	`purchases.purchased_at`
PAB Quiz	`attachment_style`	Extracted via PAB inspect API: *'Anxious-Preoccupied'*, *'Dismissive-Avoidant'*, *'Fearful-Avoidant'*, or *'Secure'*.	`past_records.summary_notes`
PAB Quiz	`progress / streaks`	Formatted into textual interaction summaries for prompt context.	`past_records.summary_notes`

Security & Privacy Safeguard (Excluded Data): Passwords, payment card PAN/CVVs, banking details, physical street addresses, and phone numbers are strictly filtered out and never transmitted to or persisted in the MySQL database.

3. Storage Architecture: The MySQL `rag_users` Schema

MySQL runs locally on the host at 127.0.0.1:3306 (isolated from public ingress). The database rag_users is structured into three core relational tables with foreign-key cascade integrity.

Table Name	Primary Key & Indexes	Columns & SQL Data Types	Operational Purpose & Patterns
users	`PK: id` `UNIQUE: email`	• id VARCHAR(50) • name VARCHAR(100) • email VARCHAR(100) • created_at TIMESTAMP	**Dual-Identity Mapping:** Stores both numeric WordPress ID (e.g. '1836') and email alias rows (e.g. 'user@ex.com'). Uses ON DUPLICATE KEY UPDATE to prevent race condition duplicates.
purchases	`PK: id (AUTO_INC)` `FK: user_id → users.id`	• id INT AUTO_INCREMENT • user_id VARCHAR(50) • item_type ENUM(...) • item_title VARCHAR(255) • purchased_at TIMESTAMP	Tracks courses, books, and coaching sessions owned by the client. Enables the RAG chat engine to tailor answers based on specific products the client has purchased.
past_records	`PK: id (AUTO_INC)` `FK: user_id → users.id`	• id INT AUTO_INCREMENT • user_id VARCHAR(50) • interaction_type VARCHAR(100) • summary_notes TEXT • recorded_at TIMESTAMP	Stores human-readable narrative snippets (Order summaries, Attachment Quiz classifications, and coaching notes) injected directly into LLM prompts under clinical context tags.

The Dual-Identity Storage Strategy:
Because incoming user queries may identify a user by either their numeric WordPress ID ('1836') or their email address ('client@domain.com'), the sync system executes a dual-row write for each customer:
1. INSERT INTO users (id, name, email) VALUES ('1836', 'Jane Doe', 'jane@domain.com') ON DUPLICATE KEY UPDATE name=VALUES(name);
2. INSERT INTO users (id, name, email) VALUES ('jane@domain.com', 'Jane Doe', 'jane@domain.com') ON DUPLICATE KEY UPDATE name=VALUES(name);
This enables immediate O(1) index lookups during live chat streaming without requiring runtime string scans or joins.

Exact MySQL Production DDL:

```
CREATE TABLE users (id VARCHAR(50) PRIMARY KEY, name VARCHAR(100) NOT NULL, email VARCHAR(100) UNIQUE, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
CREATE TABLE purchases (id INT AUTO_INCREMENT PRIMARY KEY, user_id VARCHAR(50) NOT NULL, item_type ENUM('course','book','session','consultation') NOT NULL, item_title VARCHAR(255) NOT NULL, purchased_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE);
CREATE TABLE past_records (id INT AUTO_INCREMENT PRIMARY KEY, user_id VARCHAR(50) NOT NULL, interaction_type VARCHAR(100), summary_notes TEXT, recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE);
```

4. APIs, Protocols & Synchronization Mechanisms

Data movement between Levamo and MySQL operates across three distinct lifecycles to ensure real-time responsiveness, reconciliation resiliency, and on-demand fallback.

Mechanism	Direction & Protocol	Endpoint / Interface	Authentication & Security	Trigger & Frequency
1. Real-Time Webhook Push	Levamo → FastAPI HTTP POST (JSON)	`/api/webhooks/woocommerce` `(Aliases: /sync, /reconcile)`	HMAC-SHA256 signature in header x-wc-webhook-signature via WC_WEBHOOK_SECRET.	Instant on WooCommerce events: customer.* and order.*.
2. Nightly Reconciliation ETL	FastAPI Host → Levamo HTTP GET (REST API)	`/wp-json/wc/v3/customers` `/wp-json/wc/v3/orders`	HTTP Basic Auth using WooCommerce Application Passwords (WORDPRESS_USER / WORDPRESS_APP_PASSWORD).	Cron scheduled every night at 02:00 UTC via sync_wordpress_to_mysql.py.
3. On-Demand Dynamic Lookup	FastAPI Host → Levamo HTTP GET (JSON)	`/wp-json/pab/v1/inspect/profile?user_id={id}&fresh;=true`	Custom API Key passed in header: X-API-Key: {PAB_API_KEY}	Dynamic fallback when a user initiates a chat session and is absent from local DB. Cached for 300s in RAM.
4. Database Read/Write	FastAPI / Scripts → MySQL TCP Socket 3306	`mysql://127.0.0.1:3306/rag_users`	MySQL native auth via rag_user + MYSQL_PASSWORD. Bound to localhost.	Sub-millisecond writes during webhook/ETL ingestion; read on every authenticated chat query.

5. End-to-End Ingestion Sequence & Execution Summary

The step-by-step logic summarizes how an event translates into active MySQL RAG context:

- **Step 1 (Checkout Event):** Customer purchases a coaching session or course on the WordPress site. WooCommerce fires an HTTP POST to /api/webhooks/woocommerce.
- **Step 2 (Cryptographic Verification):** FastAPI calculates HMAC-SHA256(body, WC_WEBHOOK_SECRET). If valid, processing continues; otherwise, rejected with HTTP 401.
- **Step 3 (Customer Upsert):** Customer ID and email are parsed. Both primary ID and email alias rows are written to users via INSERT ... ON DUPLICATE KEY UPDATE.
- **Step 4 (Line Item Parsing):** Line items are mapped to book, session, or course, and inserted into purchases with timestamps.
- **Step 5 (Historical Summary Record):** A structured narrative string (e.g., *'Order #12450: Purchased The Science of Connection'*) is recorded in past_records.
- **Step 6 (Chat Prompt Injection):** When the client initiates chat, rag_users is queried in sub-milliseconds to ground the LLM with the user's attachment style and courses.

6. Key Source Code Artifacts & Reference Matrix

- **Ingestion & Database Driver:** src/user_db.py (Contains UserDB, ingest_woocommerce_order_webhook(), and cache management)
- **Scheduled Batch Syncer:** scripts/sync_wordpress_to_mysql.py (Contains WordPressToMySQLSyncer, pagination, and retry logic)
- **Webhook HTTP Routing:** app.py (Contains woocommerce_webhook() and HMAC verification)
- **Database Schema DDL:** schema_mysql.sql (Defines users, purchases, and past_records DDL)
- **Mermaid Architecture Diagram:** docs/architecture/02_levamo_to_mysql_data_flow.mmd